

Project Proposal

Provenance: An Agentic Graph-Based Approach to Sustaining Narrative Continuity and Detecting Multi-Source Logical Friction

Project Scope

The goal of **Project Provenance** is to process long-form narratives, find conflicting details and logical inconsistencies. While LLMs excel at local text comprehension, they lack a persistent "state-tracking" mechanism to ensure consistency across hundreds of pages or disparate files. **Provenance** acts as an auditor that bridges the gap between unstructured text and structured logic. Two use cases for this project can be:

1. **Creative Narrative Continuity:** Assisting authors by tracking a "World State" (e.g., character inventory, physical attributes, and locations) to flag plot holes or continuity errors across chapters.
2. **Legal & Investigative Audit:** Addressing the **Rashomon Effect**, where different witnesses provide conflicting accounts. By aligning testimony across police reports, depositions, and transcripts, the system identifies and characterizes logical "Friction Points."

Planned Approach

We propose a **three-tier Agentic GraphRAG architecture**:

- **Cross-Context Alignment (xCoRe):** We implement a unified coreference resolution strategy to link entities across disparate contexts. This ensures that "the suspect" in Chapter 1 and "Mr. Smith" in Chapter 20 (or a separate deposition file) are mapped to a single unique node in the graph.
- **Deep Provenance Property Graph:** We utilize **Neo4j** to store knowledge in a specialized format. Every extracted assertion (edge) is enriched with metadata, including **Source Authority**, **Extraction Confidence**, and **Temporal Validity**. This allows the system to maintain "Multi-World Versioning," where conflicting claims are not overwritten but are instead stored as competing "realities" tagged with their respective origin.
- **Graph-Based Inconsistency (GraphFC):** Instead of binary text-comparison, we use **Graph-Guided Planning**. The system decomposes a verification request into a **Claim Graph** and performs subgraph matching against the **Evidence Graph**. This identifies structural contradictions (e.g., a character being in two places at once) that local-window NLI models miss.
- **Agentic Orchestration:** A central agent (orchestrated via LangGraph) iteratively explores the text to resolve "Mention Ambiguity." It uses fine-tuned, low-cost models (like **DeBERTa-v3**) for high-speed logical checks, reserving larger LLMs for final divergence synthesis and explanation.

Expected Outcome

The primary deliverable is a **Software Application featuring a Conflict Dashboard**.

- **Scientific Hypothesis:** A Graph-based state-tracking approach will demonstrate a higher **Recall** for long-range contradictions compared to vanilla LLM prompts (e.g., GPT-4).
- **Deliverables:** A visual "Friction Matrix" highlighting specific lines of text where provenance chains collide, and a benchmark report comparing **Provenance** against standard RAG baselines.

CHOOSING YOUR ROLE

To balance the workload while removing the dedicated UI role, the responsibilities have been redistributed to focus on the core technical pillars of Project Provenance: **Context**, **Knowledge**, **Logic**, and **System Integrity**.

Here is the revised 4-member role distribution:

1. The Context Architect

 **Mission:** You are the "front door" of the NLP pipeline, ensuring that every entity mentioned across 100+ pages is correctly identified and linked to a unique identity.

- **Key Responsibilities:**

- Build robust **PDF/DOCX/TXT document parsers** that preserve chapter and section structure.
- Implement the **spaCy NER pipeline** to extract entities like people, locations, and objects.
- Develop the **xCoRe (Cross-Context Coreference Resolution)** module to link "John" in Chapter 1 to "Mr. Smith" in Chapter 20.
- Apply **semantic embeddings** (sentence-transformers) to handle entity disambiguation and near-duplicate mentions.

- **Main Tech:** Python, spaCy, sentence-transformers, PyPDF2.

2. The Knowledge Engineer

 **Mission:** You transform messy, unstructured sentences into a structured, provenance-aware knowledge graph in Neo4j.

- **Key Responsibilities:**

- Design and manage the **Neo4j Labeled Property Graph (LPG)** schema.

- Implement **LLM-driven triple extraction** (Subject-Predicate-Object) using dependency parsing and few-shot prompting.
- Build the **Temporal & Spatial Extractor** to capture when and where events occur.
- Implement **Multi-World Versioning**, ensuring conflicting claims are stored as competing "realities" rather than being overwritten.
- **Main Tech:** Neo4j, Cypher, Python (py2neo), spaCy.

3. The Logic Strategist

 **Mission:** You are the "detective" who designs the algorithms to find contradictions using graph patterns and deep learning.

- **Key Responsibilities:**
 - Implement **Graph-Based Friction Checking (GraphFC)** to identify structural contradictions (e.g., a character in two places at once).
 - Write and optimize complex **Cypher queries** for the five core contradiction types: Temporal, Inventory, Location, Identity, and Attribute.
 - Integrate **DeBERTa-v3 NLI models** to perform high-speed semantic entailment and contradiction checks.
 - Develop the **Friction Scorer** to calculate the severity and confidence of detected conflicts.
- **Main Tech:** DeBERTa-v3, PyTorch, HuggingFace, Cypher.

4. The System Orchestrator & Validator

🎯 **Mission:** You glue the modules together into a functioning agentic system and provide the scientific proof that it works better than standard LLMs.

- **Key Responsibilities:**
 - Manage **Agentic Orchestration via LangGraph**, creating the stateful "Router-Planner" workflow that moves data between extraction and detection.
 - Build the **Universal Error Injector** to create synthetic test cases by systematically seeding documents with known contradictions.
 - Execute **Scientific Benchmarking** using datasets like ConflictBank and NovelQA to calculate metrics like Recall, CAR (Conflict-Aware Recall), and MR (Memorization Ratio).
 - Generate the **final JSON/CSV reports** and scientific validation reports comparing Provenance against GPT-4 baselines.
- **Main Tech:** LangGraph, OpenAI API (for baselines), scikit-learn, Python.

Balanced Workload Overview

Aspect	Context Architect	Knowledge Engineer	Logic Strategist	System Orchestrator
Primary Focus	Entity Linking (NLP)	Graph Construction	Reasoning Logic	System Flow & Proof
Complexity	★ ★ ★ Medium	★ ★ ★ ★ ★ Med-High	★ ★ ★ ★ ★ ★ High	★ ★ ★ ★ ★ Med-High
Key Output	"Golden" Entity Nodes	Property Graph	Detected Friction	Scientific Benchmark

--	--	--	--	--

Would you like me to draft a Week 1-2 action plan for these specific roles based on the project timeline?

Research

Advanced Architectures for Narrative Consistency: A Comprehensive Technical Framework for Project Provenance

The rapid proliferation of large-scale document collections and the increasing complexity of cross-document information synthesis have highlighted a critical vulnerability in current natural language processing systems: the inability to maintain longitudinal narrative consistency. As large language models are increasingly deployed to synthesize information from disparate, potentially conflicting sources, the necessity for robust provenance and inconsistency detection has become paramount. Project Provenance represents an integrated approach to this challenge, combining the structural advantages of Graph-based Retrieval-Augmented Generation (GraphRAG), the entity-linking capabilities of Cross-Document Coreference Resolution (CDCR), and the formal verification logic of graph-guided inconsistency detection. This report provides a detailed technical analysis of the state-of-the-art research in these domains, a rigorous project management plan for a specialized team of researchers, and a standardized benchmarking framework for narrative auditing.

Theoretical Framework and the Evolution of Graph-Based Retrieval

The limitations of traditional Retrieval-Augmented Generation (RAG) are primarily rooted in its reliance on linear text chunking and vector-based semantic similarity. While vanilla RAG pipelines effectively retrieve specific facts that share a semantic subspace with a query, they consistently struggle with global information organization and the preservation of hierarchical relationships between concepts. When domain knowledge is scattered across multiple documents, the absence of explicit relational links leads to fragmented retrieval, where the model fails to "connect the dots" across multi-hop reasoning chains. GraphRAG addresses this architectural deficiency by encoding heterogeneous information into a network of nodes and edges, thereby capturing the intrinsic relational structure of the data.

Architectural Paradigms in GraphRAG

Current research identifies several distinct approaches to integrating graph structures into the RAG workflow. The primary innovation lies in the transition from unstructured text indices to structured knowledge graphs that act as the retrieval source. Frameworks such as Microsoft GraphRAG utilize hierarchical community-based search to combine local and global querying, enabling the system to provide comprehensive responses to high-level queries that a chunk-based retriever would miss. Further refinements in systems like LightRAG improve scalability through dual-level retrieval and graph-enhanced indexing, while StructRAG allows the system to dynamically select optimal graph schemas tailored to specific tasks.

A particularly compelling development for Project Provenance is the concept of the Graph of Records (GoR). In this paradigm, a graph is constructed by establishing edges between retrieved

text chunks and the corresponding responses generated by the language model. This creates a historical reasoning graph that allows the model to maintain context over long-form global summarization tasks, effectively using its own past outputs as nodes in a growing knowledge network. This iterative refinement is essential for detecting inconsistencies that emerge not just between documents, but between a model's previous assertions and new evidence.

RAG Methodology	Data Organization	Retrieval Mechanism	Context Preservation
Vanilla RAG	Linear Chunks	Vector Similarity	Low (Fragmented)
Hierarchical RAG	Multi-level Trees	Structured Traversal	Medium (Structural)
GraphRAG (LPG)	Property Graphs	Subgraph Matching	High (Relational)
Graph of Records	Response-Chunk Edges	Iterative Self-Reference	High (Reasoning)
ChronoRAG	Temporal Chains	Chronological Indexing	High (Sequential)
Source ID			

Temporal and Multi-Agent Considerations

In narrative auditing, the temporal order of events is often the primary vector for identifying inconsistencies. ChronoRAG introduces a focused approach to narrative texts by explicitly capturing and maintaining the temporal order among retrieved passages. By refining dispersed information into coherent and structured passages that preserve narrative flow, ChronoRAG ensures that the model does not conflate events occurring at different points in a timeline.

Complementing these structural innovations is the move toward multi-agent orchestration. The Multi-Agent GraphRAG framework utilizes a modular system of LLM agents to generate and refine Cypher queries for Labeled Property Graph (LPG) databases. This system includes

specialized agents for query generation, entity verification, and feedback-driven refinement, ensuring that the retrieved subgraphs are both syntactically correct and semantically aligned with the user's intent. This agentic approach is critical for Project Provenance, as it allows for specialized auditing agents to interrogate the graph from multiple logical perspectives.

Cross-Document Coreference Resolution as a Semantic Anchor

The efficacy of any graph-based inconsistency detection system is fundamentally limited by its ability to resolve entities across different contexts. Cross-Document Coreference Resolution (CDCR) provides the necessary semantic bridge, identifying when mentions in separate documents refer to the same entity or event. Without robust CDCR, a graph-based system would create duplicate nodes for the same entity, leading to fragmented subgraphs and a failure to detect contradictions between them.

Challenges in Cross-Document Resolution

Unlike within-document coreference, where mentions are linked by a rich and coherent narrative context, cross-document mentions lack shared discourse markers. This challenge is amplified in long-form narratives where descriptions of the same entity may evolve or diverge significantly. To address this, recent methodologies like Cross-Document Discourse Coherence Enhancement (CD-DCE) focus on enhancing the coherence between mentions by selecting and inserting contextually relevant sentences between them, thereby creating a unified representation space for resolution.

The xCoRe framework represents a significant step toward unifying the challenges of long-document and cross-document coreference. By adopting a three-step pipeline—identifying mentions, creating clusters within individual context windows, and merging those clusters across contexts—xCoRe addresses the memory and architectural constraints that typically limit coreference systems to short texts. This unified "cross-context" approach enables joint training on shared resources, which has been shown to improve performance on the traditionally difficult cross-document task.

Synergy and Specialized Models

The SECURE framework further refines the CDCR process by leveraging a collaborative approach between general-purpose large language models and task-specific small language models (SLMs). SECURE focuses on processing mentions individually, which is significantly more efficient than previous methods that relied on quadratic mention-pair comparisons. This efficiency is crucial for Project Provenance, which must handle large-scale corpora where pairwise comparisons would become computationally prohibitive.

The choice of underlying language models also plays a vital role. DeBERTa-v3 has emerged as a state-of-the-art encoder for these tasks, utilizing disentangled attention mechanisms to capture deeper contextual information. When fine-tuned on Natural Language Inference (NLI) datasets, such as the `deberta-base-long-nli` model, these encoders can accurately determine whether two pieces of text entail or contradict each other, providing the logical foundation for both

coreference and inconsistency detection.

CDCR Framework	Core Mechanism	Scale Capacity	Primary Innovation
CD-DCE	Context Enhancement	Medium	Discourse Coherence Theory
xCoRe	Cross-context Merging	Very High	Unified Long/Cross-doc Pipeline
SECURE	LLM-SLM Collaboration	High	Efficient Individual Mention Processing
Lawformer	Longformer Architecture	High	Domain-Specific (Legal) Pre-training
Source ID			

The integration of these CDCR techniques into the Project Provenance graph allows for the creation of "golden" entity nodes that serve as the ground truth for all mentions across the corpus. This ensures that when the inconsistency detection module identifies a conflict, it is correctly attributed to the intended entity.

Graph-Guided Inconsistency Detection and Fact-Checking Logic

The final stage of the Provenance framework is the formal detection of inconsistencies through graph-guided reasoning. Traditional LLM-based fact-checking often fails because models decompose claims into isolated textual sub-claims, which ignores the relational constraints between them and increases the ambiguity of mentions. Graph-based frameworks like GraphFC and GraphCheck resolve these issues by explicitly encoding claims and evidence as directed graphs of atomic triplets.

The GraphFC Verification Cycle

GraphFC converts natural language claims into a directed graph G_c consisting of triplets $(s,$

r, o)\$, where entities can be known (grounded in the claim) or unknown (placeholders to be resolved). The verification process is structured around three key components:

1. **Graph Construction:** Independent graphs are built for the claim and the provided evidence corpus, reducing the impact of irrelevant context that often impairs standard LLM verification.
2. **Graph-Guided Planning:** This component determines the optimal logical sequence for verification by prioritizing triplets based on their resolution state. Triplets with two known entities are verified first, as they provide the anchors for grounding unknown placeholders in subsequent steps.
3. **Graph-Guided Checking:** This involves direct subgraph matching for grounded triplets and graph completion for those with unknown entities, where the system searches the evidence graph to propose concrete grounding.

A claim is formally supported in GraphFC if and only if there exists a mapping between the claim graph and the evidence graph that satisfies all relational constraints. This ternary-level verification prevents the "insufficient decomposition" problem where complex claims are oversimplified into broad, unverifiable statements.

Explainability and Multi-hop Verification

The use of Graph Neural Networks (GNNs) in frameworks like GraphCheck further enhances the ability to capture multi-hop reasoning chains. By processing knowledge graphs as soft prompts, GraphCheck allows LLMs to incorporate structured knowledge more effectively than through raw text alone. This methodology provides fine-grained explainability, as the system can identify the specific entity relationships and paths it focused on during the verification process.

For long-document consistency, systems like UIEFID utilize self-adaptive structured schemas to guide LLMs in extracting unified information from documents and summaries. By pinpointing the origins of inconsistencies within this structured information, the system can provide substantiating evidence for why a summary or report deviates from its source. This is particularly critical in domains like legal and medical research, where subtle factual errors can have severe consequences.

Technical Orchestration and Agentic Systems

Managing the complexity of GraphRAG and CDCR requires a sophisticated orchestration layer. LangGraph has emerged as the industry standard for building these modular, stateful workflows among LLM agents. Unlike linear pipelines, LangGraph models the verification process as a dynamic decision loop where agents can pause, retry, and re-plan based on intermediate findings.

State Management and Persistence

The core of a LangGraph system is its StateGraph, which maintains a strongly typed state object that persists across node transitions. This state typically includes the current collection of retrieved documents, the set of resolved entities, and the execution history. Persistence allows the system to handle long-running processes, such as auditing a massive document corpus, where the

workflow may need to pause for external human review or wait for a document re-submission.

Orchestration Pattern	Application in Provenance	Mechanism
Router-Planner-Coordinator	Task Allocation	Agents decompose complex audits into specific verification steps.
Human-in-the-Loop (HIL)	Regulatory Compliance	Workflow pauses at high-stakes nodes for human approval.
Multi-Agent Refinement	Query Optimization	One agent generates Cypher, another executes, a third evaluates.
Scatter-Gather	Parallel Processing	Distributing different documents to multiple agents for extraction.
Source ID		

In Project Provenance, this orchestration allows for a "jury" of LLMs to score results, ensuring that verification is not dependent on a single model's biases. The observability provided by LangGraph enables per-agent logging and diagnostics, which is essential for auditing the auditor—ensuring that the consistency detection itself is consistent and transparent.

Benchmarking Consistency and Knowledge Conflicts

The evaluation of a system designed to detect inconsistencies must itself be rigorous. Traditional benchmarks often fail to capture the nuance of knowledge conflicts, where a model's internal parametric knowledge may contradict newly retrieved evidence. ConflictBank addresses this by providing a comprehensive testbed for misinformation, temporal discrepancies, and semantic divergences.

Conflict Evaluation Metrics

ConflictBank introduces several novel metrics for quantifying model behavior in the face of conflicting information. The Memorization Ratio (MR) measures how often a model defaults to its internal training data when presented with contradictory evidence, while Conflict-Aware Recall (CAR) assesses the model's ability to identify all valid, albeit conflicting, answers to a

question.

Benchmark	Primary Task	Metric Focus	Complexity
ConflictBank	Knowledge Conflict Detection	MR, CAR	7.45M claim-evidence pairs
NovelQA	Long-range Narrative QA	Multi-hop F1	100k+ token context
Loong	Multi-document QA	Relational Accuracy	Realistic long-dependencies
NATCONFQA	Conflict-aware MAQA	$F1_{\{conf\}}$	Labeling conflicting answer pairs
Source ID			

NovelQA provides a specialized environment for assessing narrative continuity over extremely long contexts, such as full novels. These benchmarks reveal that even the most advanced models struggle with multi-hop reasoning and detail-oriented questions when information is scattered throughout a 200,000-token input. This underscores the necessity of the graph-based indexing and retrieval strategies central to Project Provenance.

Project Provenance Implementation Plan

The following project plan outlines a ten-week development cycle for a team of four NLP/DL students. The focus is strictly on the technical implementation of the GraphRAG, CDCR, and inconsistency detection components.

Technical Role Allocation

Roles are assigned based on technical specialization to ensure comprehensive coverage of the system's requirements.

- **Graph Systems Architect:** Responsible for the design and implementation of the Labeled Property Graph (LPG) schema in Neo4j/Memgraph. This role focuses on the Text-to-Cypher generation pipeline, multi-world versioning for conflicting narratives, and

the overall retrieval logic. Technical focus: Neo4j, Cypher, graph database optimization, and LangGraph state management.

- **Coreference Resolution Specialist:** Focused on the CDCR pipeline. This involves mention identification, intra/cross-context clustering, and discourse coherence enhancement. This role will also manage the fine-tuning of encoders like DeBERTa-v3 for logical consistency checks. Technical focus: SECURE, xCoRe, Hugging Face Transformers, and NLI-based verification.
- **Consistency Analysis Developer:** Dedicated to the implementation of the GraphFC and GraphCheck verification logic. This includes claim decomposition into triplets, graph-guided planning, and the development of the priority scoring function. Technical focus: PyTorch, GNNs, GraphFC verification logic, and multi-hop reasoning modules.
- **Evaluation & Research Engineer:** Responsible for the benchmarking pipeline and the scientific validation of the system. This role will integrate ConflictBank, NovelQA, and other narrative benchmarks, calculating the formal metrics (MR, CAR, $F1_{\{conf\}}$) and managing experiment tracking. Technical focus: Python, benchmarking frameworks, statistical validation, and research reporting.

Timeline and Weekly Deliverables

The project spans from mid-February to the end of April, with a focus on incremental integration and rigorous testing.

Month 1: Foundation and Knowledge Extraction (Feb 15 - Mar 14)

- **Week 1 (Feb 15-21): System Architecture and Data Ingestion.**
 - Objective: Finalize technical specifications and initialize the graph database environment.
 - Deliverables: Graph schema definition (Neo4j) and initialized development environments for all roles.
 - Milestone: Successful ingestion of the primary document corpus into the graph backend.
- **Week 2 (Feb 22-28): Knowledge Triple Extraction.**
 - Objective: Implement the LLM-driven triplet extraction module for both claims and documents.
 - Deliverables: Extraction pipeline prototype using few-shot prompting and UIE patterns.
 - Milestone: Alpha version of the claim-to-graph conversion module.
- **Week 3 (Mar 1-7): Entity Resolution and Coreference Baseline.**
 - Objective: Deploy the initial mention identification and clustering system.
 - Deliverables: Baseline CDCR pipeline showing entity linking across a subset of documents.
 - Milestone: Completion of the xCoRe intra-context clustering module.
- **Week 4 (Mar 8-14): Agentic Orchestration Setup.**
 - Objective: Configure LangGraph for macro-level workflow orchestration.
 - Deliverables: StateGraph implementation for the Router-Planner workflow.

- Milestone: First end-to-end "Query-Retrieve-Extract" loop.

Month 2: Inconsistency Logic and Integration (Mar 15 - Apr 11)

- **Week 5 (Mar 15-21): Graph-Guided Planning Implementation.**
 - Objective: Develop the logical sequence for triplet verification.
 - Deliverables: Priority function $\rho(t)$ implementation and ordered verification list generation.
 - Milestone: Successful prioritization of triplets based on resolution status.
- **Week 6 (Mar 22-28): Cross-Document Verification and Matching.**
 - Objective: Implement the Graph Match and Graph Completion sub-tasks.
 - Deliverables: Subgraph matching module for grounded triplets and completion logic for placeholders.
 - Milestone: Detection of simple factual contradictions in controlled document pairs.
- **Week 7 (Mar 29 - Apr 4): Temporal and Versioning Logic.**
 - Objective: Integrate ChronoRAG and multi-world versioning for narrative auditing.
 - Deliverables: Temporal edge implementation and entity versioning nodes in Neo4j.
 - Milestone: Capability to retrieve "snapshots" of entity states over time.
- **Week 8 (Apr 5-11): Full Pipeline Integration.**
 - Objective: Synchronize CDCR, GraphRAG, and Inconsistency modules into a single workflow.
 - Deliverables: Integrated Project Provenance system.
 - Milestone: System pass on the small-scale multi-hop reasoning test set.

Month 3: Evaluation and Benchmark Reporting (Apr 12 - Apr 30)

- **Week 9 (Apr 12-18): Large-Scale Benchmarking.**
 - Objective: Execute the evaluation pipeline on ConflictBank and NovelQA datasets.
 - Deliverables: Comprehensive performance report including MR, CAR, and narrative accuracy metrics.
 - Milestone: Validation of system superiority over vanilla RAG baselines on multi-hop tasks.
- **Week 10 (Apr 19-30): Final Optimization and Scientific Report.**
 - Objective: Refine model parameters and finalize the benchmark report in scientific format.
 - Deliverables: Final project repository and a publication-quality research report.
 - Milestone: Final project presentation and defense of the Provenance framework.

Scientific Benchmark Report Technical Specification

To ensure a standard scientific format, the benchmark report will follow a rigorous structure, focusing on empirical validation and error analysis.

Standardized Evaluation Protocol

Every experiment must be conducted under a controlled setting, specifying the underlying model backbone (e.g., Llama-3, GPT-4o, or DeBERTa-v3) and the specific graph database configuration.

- Task Definition:** Formally define the fact-checking task as a mapping problem between G_c and G_e , adhering to Assertion 1 of the GraphFC framework.
- Dataset Inventory:** Detailed description of the datasets used, including the number of documents, total tokens, and the complexity of the coreference chains.
- Ablation Study:** Quantification of the contribution of each component (e.g., the performance impact of removing the CDCR step or the graph completion module).
- Error Analysis:** Qualitative analysis of common failure modes, such as coreference propagation errors or latent entity infilling failures.

Quantitative Metrics and Mathematical Formulation

The report will prioritize the following formal metrics:

- Memorization Ratio (MR):** $MR = \frac{\text{Count}(\text{DefaultToParametric})}{\text{Total}(\text{ConflictInstances})}$, quantifying the system's reliance on grounded evidence versus internal bias.
- Conflict-Aware Recall (CAR):** Evaluates the comprehensiveness of conflict identification, ensuring that the system does not overlook subtle discrepancies in multi-answer scenarios.
- Pearson Correlation:** Used to evaluate the alignment between the system's consistency scores and human expert judgments on narrative continuity.
- CoNLL F1:** The standard metric for coreference resolution, measuring the overlap between the system's clusters and the ground truth.

Metric Class	Specific Metric	Domain	Benchmark Target
Verification	Veracity Accuracy	Fact-Checking	FEVEROUS, HOVER
Conflict	CAR / MR	Knowledge Integrity	ConflictBank

Reasoning	Multi-hop F1	Narrative Logic	NovelQA, Loong
Coreference	MUC / \$B^3\$	Entity Linking	ECB+, GVC
Efficiency	Token Utilization	System Overhead	Custom Audits
Source ID			

Strategic Synthesis of Narrative Provenance

The integration of GraphRAG, CDCR, and graph-guided inconsistency detection within Project Provenance represents a paradigm shift from keyword-based search to structural reasoning. The methodology moves away from the "black box" nature of typical LLM retrieval, providing a transparent and auditable chain of evidence for every identified inconsistency.

By leveraging Multi-Agent GraphRAG systems, the project enables specialized agents to interact with property graphs derived from complex technical data, such as Industry Foundation Classes (IFC) or medical abstracts. This capability transforms the system from a general-purpose researcher into a domain-specific expert capable of detecting subtle nuances in professional narratives. The use of disentangled attention in models like DeBERTa-v3 ensures that the underlying representations are robust enough to handle the linguistic variability of cross-document mentions.

Furthermore, the implementation of LangGraph's state management ensures that the auditing process is both scalable and deterministic. Every transition in the verification workflow is logged, allowing for the reconstruction of the model's "thought process". This transparency is essential for the future of automated fact-checking, where the goal is not just to provide a veracity label, but to provide a verifiable, context-aware justification grounded in the provenance of the information. As the digital news ecosystem continues to fragment and misinformation becomes more sophisticated, the structural and logical rigor provided by Project Provenance will serve as a critical foundation for information integrity.

The successful execution of this ten-week project will demonstrate that narrative consistency can be formally managed through the synthesis of graph-structured data and agentic reasoning. The resulting framework will provide a robust methodology for auditing long-form, multi-source document collections, ensuring that the insights derived from these corpora are grounded in evidence and resistant to the hallmarks of large language model hallucinations. Through the systematic application of CDCR and graph-guided verification, Project Provenance establishes a new standard for narrative integrity in the era of generative artificial intelligence.

SRS

PROJECT PROVENANCE

Software Requirements Specification (SRS)

An Agentic Graph-Based Approach to
Sustaining Narrative Continuity and Detecting Multi-Source Logical Friction

Version 1.0

February 12, 2026

Project Duration: February 15, 2026 - April 30, 2026

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a comprehensive description of Project Provenance, a novel system designed to detect logical inconsistencies and narrative continuity errors in long-form textual documents. The document is intended for:

- Development team members responsible for implementing the system
- Academic supervisors and evaluators
- Future maintainers and researchers

1.2 Scope

Project Provenance is an NLP-powered software system that employs a three-tier agentic GraphRAG architecture to identify contradictions in narrative documents. The system will:

- Process documents ranging from 10 to 100+ pages
- Extract entities, relationships, and temporal/spatial information
- Build a provenance-aware knowledge graph using Neo4j
- Detect five core contradiction types: Temporal, Inventory, Location, Identity, and Attribute
- Provide a visual Conflict Dashboard for exploration and validation

Primary Use Cases:

1. **Creative Narrative Continuity:** Assisting authors in tracking world state (character inventory, physical attributes, locations) to flag plot holes across chapters
2. **Legal & Investigative Audit:** Addressing the Rashomon Effect by aligning conflicting witness testimonies across police reports, depositions, and transcripts

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
GraphRAG	Graph-based Retrieval-Augmented Generation
xCoRe	Cross-Context Reference Resolution
GraphFC	Graph-based Friction Checking
NLI	Natural Language Inference
Triple	Subject-Predicate-Object relationship (e.g., [John]-[owns]->[car])
Provenance	The source, timestamp, and confidence metadata attached to each extracted fact
Friction Point	A detected logical inconsistency between two or more assertions
Multi-World Versioning	Storing conflicting claims as separate graph branches tagged by source

1.4 References

- [Neo4j Graph Database Documentation \(v5.x\)](#)
- [LangGraph Multi-Agent Framework](#)
- [DeBERTa-v3 Fine-tuning Guide](#)
- [spaCy v3 NER and Coreference Resolution](#)

2. Overall Description

2.1 Product Perspective

Project Provenance is a standalone research prototype that bridges unstructured text analysis with structured graph reasoning. Unlike traditional RAG systems that operate on isolated document chunks, Provenance maintains a global state representation through a property graph, enabling long-range contradiction detection.

System Architecture:

Layer	Components
Input Layer	Document Parser (PDF/DOCX/TXT), Text Chunker, Metadata Extractor
NLP Layer	spaCy NER, Dependency Parser, xCoRe Module, Temporal Extractor
Graph Layer	Neo4j Database, Triple Builder, Provenance Tagger, Multi-World Version Manager
Reasoning Layer	GraphFC Module, Cypher Query Engine, DeBERTa-v3 NLI Validator, Friction Scorer
Presentation Layer	Streamlit Dashboard, Conflict Visualizer, Report Generator, Graph Explorer

2.2 Product Functions

2.2.1 Document Ingestion

- Accept PDF, DOCX, and TXT files
- Extract text while preserving chapter/section structure
- Chunk documents into sentence-level segments with metadata

2.2.2 Entity and Relationship Extraction

- Identify entities (people, objects, locations) using spaCy NER
- Extract subject-predicate-object triples using dependency parsing
- Resolve coreferences across document contexts (xCoRe module)
- Attach provenance metadata: source text, page number, timestamp, confidence score

2.2.3 Graph Construction

- Store entities as nodes with properties (name, type, attributes)
- Store relationships as edges with provenance properties
- Implement multi-world versioning for conflicting facts
- Support temporal validity windows for time-sensitive assertions

2.2.4 Contradiction Detection (GraphFC)

- Perform graph queries to identify five contradiction types
- Use DeBERTa-v3 for semantic entailment checking
- Generate friction scores (0-1) indicating contradiction severity
- Trace provenance chains to source text locations

2.2.5 Visualization and Reporting

- Display Friction Matrix with contradiction counts by type
- Show side-by-side text comparison for each conflict
- Enable graph visualization of provenance chains
- Export detailed JSON report with all detected inconsistencies

2.3 User Classes and Characteristics

User Type	Technical Level	Primary Goal	Interface Needs
Creative Writer	Non-technical	Find plot holes in manuscript	Simple upload, visual results
Legal Analyst	Domain expert	Identify witness conflicts	Detailed reports, provenance trails
NLP Researcher	Highly technical	Benchmark system performance	API access, metric dashboards

2.4 Design and Implementation Constraints

- **Timeline:** 10-week development period (mid-February to end of April)
- **Team Size:** 4 members with balanced NLP workload distribution
- **Technology Stack:** Python 3.9+, Neo4j, LangGraph, spaCy, HuggingFace Transformers
- **Performance:** Process 20-page documents within 5 minutes on CPU-only infrastructure
- **Accuracy:** Detect 90% of injected errors and 70% of organic contradictions

3. Specific Requirements

3.1 Functional Requirements

ID	Requirement	Priority
FR-01	System shall accept PDF, DOCX, and TXT files up to 100 pages	High
FR-02	System shall extract entities with minimum 80% precision	High
FR-03	System shall resolve coreferences across at least 10-page document spans	High
FR-04	System shall store provenance metadata (source, timestamp, confidence) for every triple	High
FR-05	System shall detect Temporal, Inventory, Location, Identity, and Attribute contradictions	High
FR-06	System shall display contradictions in a web-based dashboard with side-by-side text comparison	Medium
FR-07	System shall export results as JSON and CSV reports	Medium
FR-08	System shall provide API endpoints for programmatic access (researcher use case)	Low

3.2 Contradiction Type Specifications

Type	Description	Example
Temporal	Entity in two places at same time, or events ordered impossibly	'John was in Paris on May 1st' vs 'John was in Tokyo on May 1st'
Inventory	Entity gains/loses object without explanation	'She had a gun' → later 'She searched for a weapon'
Location	Immovable object changes location	'The cafe is on Main St' vs 'walked to the cafe on Oak Ave'
Identity	Person incorrectly referred to by wrong name	'Detective Jane Smith' → later called 'Detective John Smith'
Attribute	Physical characteristic changes (eye color, height, etc.)	'He has blue eyes' vs 'His brown eyes narrowed'

3.3 Non-Functional Requirements

Category	Requirement
Performance	Process 20-page document in ≤ 5 minutes (CPU-only). Graph construction time ≤ 30 seconds for 100 triples.
Scalability	Support documents up to 100 pages. Graph database should handle at least 10,000 nodes and 50,000 edges.
Usability	Dashboard should be intuitive for non-technical users. No more than 3 clicks to view contradiction details.
Reliability	System should handle malformed input gracefully (corrupted PDFs, non-English text). Provide clear error messages.
Maintainability	Code must be modular with clear separation of concerns. Comprehensive unit test coverage ($\geq 70\%$). Detailed documentation for all modules.

4. System Features

4.1 Cross-Context Reference Resolution (xCoRe)

Description: Resolves entity mentions across document boundaries to ensure consistent graph representation.

Inputs: Sentence-level chunks with NER annotations

Processing:

3. Use spaCy's neural coref to cluster mentions
4. Apply string similarity (Levenshtein distance) for near-duplicates
5. Use semantic embeddings (sentence-transformers) for ambiguous cases
6. Assign confidence scores to each resolution

Outputs: Unified entity ID mappings (mention → canonical entity)

4.2 Graph-Based Friction Checking (GraphFC)

Description: Detects structural and semantic contradictions via graph pattern matching.

Inputs: Populated Neo4j graph with provenance metadata

Processing:

7. Execute Cypher queries for each contradiction type
8. Extract candidate contradiction pairs
9. Use DeBERTa-v3 NLI model to validate semantic opposition
10. Compute friction score based on confidence and provenance strength

Outputs: List of friction points with source text references

4.3 Conflict Dashboard

Description: Web-based interface for exploring and validating detected contradictions.

Features:

- Friction Matrix: heatmap showing contradiction counts by type and document section
- Detail View: side-by-side text snippets with highlighted conflicting phrases
- Provenance Chain Visualization: interactive graph showing fact dependencies
- Export Functionality: JSON and CSV reports

5. Validation Strategy

5.1 Graph Accuracy Validation

To ensure the knowledge graph accurately represents source text, we implement a three-stage validation pipeline:

Stage	Method	Metric
1. Synthetic Benchmark	Run extraction on clean document with known ground-truth graph	Triple-level Precision/Recall
2. Auditor Agent	Second model verifies each triple against source text (Yes/No)	False edge detection rate
3. Soft Matching	Tag low-confidence triples as hypotheses; require human review	Confidence score distribution

5.2 Error Injection Framework

The Universal Error Injector creates synthetic test cases with known contradictions:

- Takes a clean narrative document as input
- Systematically injects 5-10 errors of each type (Temporal, Inventory, etc.)
- Maintains ground truth mapping of error locations and types
- Runs Provenance system and calculates Precision/Recall metrics

5.3 Success Metrics

- **Triple Extraction Accuracy:** $\geq 85\%$ precision and $\geq 80\%$ recall compared to manual annotation
- **Injected Error Detection:** $\geq 90\%$ recall (detect 9 out of 10 synthetic contradictions)
- **Organic Error Detection:** $\geq 70\%$ recall on real-world test documents
- **False Positive Rate:** $\leq 20\%$ (precision $\geq 80\%$)

6. Appendices

6.1 Technology Stack

Component	Technology	Version
Programming Language	Python	3.9+
NLP Framework	spaCy	3.7+
Graph Database	Neo4j	5.x
Agentic Orchestration	LangGraph	Latest
NLI Model	DeBERTa-v3-base-mnli	HuggingFace
Web Framework	Streamlit	1.30+
Visualization	Plotly, NetworkX	Latest

6.2 Sample Cypher Query (Temporal Contradiction)

```
MATCH (e:Entity)-[r1:RELATION]->(target)WHERE r1.type = 'located_at' AND r1.timestamp IS NOT
NULLMATCH (e)-[r2:RELATION]->(different_target)WHERE r2.type = 'located_at' AND r2.timestamp IS NOT
NULL AND r1.timestamp = r2.timestamp AND target <> different_targetRETURN e.name, r1.source_text,
r2.source_text, r1.timestamp
```

6.3 Glossary of Edge Types

Edge Type	Example
OWNS	[John] -OWNS-> [Car]
LOCATED_AT	[John] -LOCATED_AT-> [Paris] {timestamp: '2024-05-01'}
HAS_ATTRIBUTE	[Sarah] -HAS_ATTRIBUTE-> [blue eyes]
MENTIONED_IN	[Detective] -MENTIONED_IN-> [Report_A], [Report_B]

Timeline - non-divided

This unified 6-week timeline focuses on the evolution of the **Provenance** system from raw text ingestion to a scientifically validated friction-detection engine. Instead of individual tasks, this follows the logical progression of the software architecture.

Week 1: The Ingestion & Schema Foundation

The focus is on building the "skeleton" of the system—preparing the data and the place where it will live.

1. **Environment Setup:** Initialize the Python environment with spaCy, Neo4j, and LangGraph.
2. **Document Chunking:** Build a structural parser that breaks 100+ page documents into manageable segments while retaining "Metadata Anchors" (Chapter, Page, Paragraph).
3. **Graph Schema Design:** Define the Neo4j Labeled Property Graph (LPG) structure, creating nodes for **Entity** and **Event**, and edges for **RELATION**.

Week 2: Triple Extraction & Identity Resolution

The system begins to "read" and understand who the characters are and what they are doing.

1. **Pipeline Construction:** Implement the NLP pipeline using spaCy to extract (Subject, Predicate, Object) triples from the text.
2. **The xCoRe Module:** Deploy the **Cross-Context Coreference Resolution**. This ensures that "The Protagonist" in Chapter 1 and "John Doe" in Chapter 5 are merged into a single unique node in the graph.
3. **Initial Ingestion:** Run the first 20 pages of a narrative through the pipeline to populate the graph with its first set of linked facts.

Week 3: World-State Tracking & Multi-World Logic

The system begins to track movement, time, and conflicting accounts (the "Rashomon Effect").

1. **Temporal/Spatial Anchoring:** Assign timestamps and location tags to every relationship in the graph.
2. **Multi-World Versioning:** Configure the database to store conflicting claims as separate "Versions" or "Beliefs" rather than overwriting old data with new data.
3. **Inventory Tracking:** Implement logic to track the "State" of objects (e.g., if a character picks up a key, the graph must reflect the key is now "held_by" that character).

Week 4: The Friction Detection Engine

The core "Audit" functionality is built to identify where the narrative breaks down.

1. **GraphFC Implementation:** Develop the **Graph-Based Friction Checking**

algorithms using Cypher queries to find structural "impossibilities" (e.g., a character being in two cities at the same time).

2. **NLI Integration:** Plug in the **DeBERTa-v3 model** to handle semantic contradictions that the graph might miss (e.g., "The door was locked" vs. "He walked through the open door").
3. **Friction Scoring:** Create a weighted scoring system that flags conflicts as "High," "Medium," or "Low" confidence based on both graph patterns and NLI results.

Week 5: Agentic Orchestration & Error Seeding

The components are tied together into a self-running agent, and the system is intentionally challenged.

1. **LangGraph Workflow:** Build the "Router-Planner" loop. This allows the system to autonomously decide when it needs to re-examine a previous chapter to verify a new fact.
2. **The Universal Error Injector:** Create a test suite of "corrupted" documents. Systematically inject 5–10 errors of each type (Temporal, Inventory, Identity) into a clean story.
3. **System Stress Test:** Run the Provenance agent against the corrupted documents to see if the Friction Engine can successfully flag the injected "plot holes."

Week 6: Benchmarking & Final Validation

The final week is dedicated to scientific proof and performance metrics.

1. **Performance Metrics:** Calculate the Precision and Recall for triple extraction (target: $\geq 85\%$) and error detection (target: $\geq 90\%$ for synthetic errors).
2. **GPT-4 Baseline Comparison:** Run the same documents through a standard GPT-4 prompt ("Find the contradictions in this story") and compare its performance against the Provenance system.
3. **Reporting & Visualization:** Finalize the dashboard to visualize the "Friction Points" within the Neo4j graph and generate the final audit report.

Summary of Key Milestones

Week	Phase	Primary Deliverable
1	Foundation	Operational Neo4j Schema & Document Parser

2	Extraction	Functional xCoRe (Entity Linking) Pipeline
3	State	Multi-World Versioning for conflicting facts
4	Logic	The Friction Engine (Cypher + DeBERTa)
5	Agentic	LangGraph Orchestration & Error Injection Test
6	Proof	Scientific Benchmark Report vs. GPT-4

Timeline Divided

Week	Context Architect (Extraction)	Knowledge Engineer (Graph)	Logic Strategist (Reasoning)	System Orchestrator (Integration)
1	Interface: Agree on Triple JSON schema.	Schema: Finalize Neo4j node/edge properties.	Baseline: Set up DeBERTa environment.	Setup: Initialize LangGraph & Repo architecture.
2	Parsing: Build PDF/DOCX structural parsers.	Ingestion: Create scripts to load JSON into Neo4j.	GraphFC: Draft Cypher queries for "Impossible Location" logic.	Stubs: Create mock "Stub" modules for E2E flow.
3	Coreference: Implement xCoRe for cross-chapter links.	Multi-World: Build versioning/branching logic.	NLI: Fine-tune DeBERTa on narrative contradiction samples.	Orchestration: Build the stateful Router/Planner logic.
4	Scaling: Run extraction on the full 100+ page doc.	Refinement: Optimize graph queries for speed.	Friction: Integrate Cypher + NLI into a single score.	First E2E: Replace Stubs with real modules for testing.
5	Optimization: Clean up "noisy" entity extractions.	Visuals: Build graph visualization (Plotly/NetworkX).	Error Injection: Use the Injector to	Testing: Conduct full system stress tests with injected errors.

			find logic gaps.	
6	Reporting: Build the source-citation generator.	Finalization: Ensure graph integrity & constraints.	Science: Compare results against GPT-4/Claude.	Delivery: Finalize the validation report and documentation.